Binary Search Problems and Solutions:

1. First and Last Occurrence in a Sorted Array

Problem:
Find the first and last position of a given target element in a sorted array.
Return {-1, -1} if the element is not found.

Approach 1: Linear Search (Worst)
- Description: Iterate through the array to find the first and last occurrence of the target.
- Time Complexity: O(n)
- Space Complexity: O(1)

- Pseudocode:

```
vector findFirstLast(vector& nums, int target) {
int first = -1, last = -1;
for (int i = 0; i < nums.size(); i++) {
if (nums[i] == target) {
if (first == -1) first = i;
last = i;
}
}
return {first,
```

last};
}

Approach 2: Binary Search (Optimal)
- Description: Use binary search to find the first occurrence, and then another binary search to find the last occurrence.
- Time Complexity: O(log n)
- Space Complexity: O(1)

Pseudocode:
vector findFirstLast(vector& nums, int target) {
int first = -1, last = -1;
int low = 0, high = nums.size() - 1;

// Find first occurrence
while (low <= high) {
int mid = low + (high - low) /2;
if (nums[mid] >= target) high = mid - 1;
else low = mid + 1;
if (nums[mid] == target) first =

```
mid;
}


low = 0, high = nums.size() - 1;
// Find last occurrence
while (low <= high) {
int mid = low + (high - low)  /2;
if (nums[mid] <= target) low = mid + 1;
else high = mid - 1;
if (nums[mid] == target) last = mid;
}

return {first, last};
}
```

Dry Run:

Input: nums = {1, 2, 2, 2, 3, 4}, target = 2

First Binary Search:

mid = 2, nums[mid] = 2 → first = 2

Adjust high = 1

mid = 0, nums[mid] = 1

Adjust low = 1

Output: First occurrence =

1

Second Binary Search:

mid = 2, nums[mid] = 2 → last = 2

Adjust low = 3

mid = 3, nums[mid] = 2 → last = 3

Output: Last occurrence = 3

Output:

{1, 3}

2. Find Peak Element

Problem:

Find a peak element in the array such that the element is greater than or equal to its neighbors. Multiple peaks are possible; return any.

Approach 1: Linear Search (Worst)

- Description: Iterate through the array to find a peak element.
- Time Complexity: O(n)
- Space Complexity: O(1)

Pseudocode:

```
int findPeakElement(vector& nums) {
for (int i = 0; i < nums.size() - 1; i++) {
if (nums[i] > nums[i + 1]) return
```

```
i;
}
return nums.size() - 1;
}
```

Approach 2: Binary Search (Optimal)

- Description: Use binary search to find a peak efficiently.
- Time Complexity: O(log n)
- Space Complexity: O(1)

Pseudocode:

```
int findPeakElement(vector& nums) {
int low = 0, high = nums.size() - 1;
while (low < high) {
int mid = low + (high - low)  /2;
if (nums[mid] < nums[mid + 1]) low = mid + 1;
else high = mid;
}
return low;
}
```

Dry Run:

Input: nums = {1, 2, 3, 1}

Binary

Search:

mid = 1, nums[mid] = 2, nums[mid + 1] = 3

Adjust low = 2

mid = 2, nums[mid] = 3, nums[mid + 1] = 1

Adjust high = 2

Output: Peak = 2

Output: 2

3. Find the Pivot Element

Problem Statement

In a rotated sorted array, the pivot element is the index where the rotation occurs. This is the largest element before the array resets.

Approach 1: Linear Search

- Idea: Traverse the array and find the index where arr[i] > arr[i + 1].
- Time Complexity: O(n)
- Space Complexity: O(1)

Pseudocode:

```
int findPivotLinear(vector& arr) {
for (int i = 0; i < arr.size() - 1; i++) {
if (arr[i] > arr[i + 1]) {
return
```

i;

}

}

return -1;

}

Dry Run:

Input: arr = [4, 5, 6, 1, 2, 3]

Traversal: Compare each pair, pivot at index 2.

Output: 2

Approach 2: Binary Search

- Idea: Use binary search to find the pivot point where arr[mid] > arr[mid + 1].

- Time Complexity: O(log n)

- Space Complexity: O(1)

Pseudocode:

```
int findPivotBinary(vector& arr) {
int low = 0, high = arr.size() - 1;

while (low < high) {
int mid = low + (high - low)  /2;
if (arr[mid] > arr[high])
```

```
{
low = mid + 1;  //Pivot is on the right
} else {
high = mid;  //Pivot is on the left
}
}
return low;
}
```

Dry Run:

Input: arr = [4, 5, 6, 1, 2, 3]

Binary Search: mid = 3, adjust bounds until low = 3.

Output: 3 (Pivot element 1)

4. Search in a Sorted and Rotated Array

Problem Statement

Find a target element in a sorted and rotated array.

Approach 1: Linear Search

- Idea: Traverse the array and check each element.
- Time Complexity: O(n)
- Space Complexity:

O(1)

Pseudocode:

```
int searchLinear(vector& arr, int target) {
for (int i = 0; i < arr.size(); i++) {
if (arr[i] == target) return i;
}
return -1;
}
```

Dry Run:

Input: arr = [4, 5, 6, 1, 2, 3], target = 2

Traversal: Target found at index 4.

Output: 4

Approach 2: Binary Search

- Idea: Perform binary search by identifying the sorted half at every step.
- Time Complexity: O(log n)
- Space Complexity:

O(1)

Pseudocode:

```
int searchBinary(vector& arr, int target) {
int low = 0, high = arr.size() - 1;
while (low <= high) {
int mid = low + (high - low)  /2;
if (arr[mid] == target) return mid;


if (arr[low] <= arr[mid]) {  //Left half is sorted
if (target >= arr[low] && target < arr[mid]) {
high = mid - 1;
} else {
low = mid + 1;
}
} else {  //Right half is sorted
if (target > arr[mid] && target <= arr[high]) {
low = mid + 1;
} else {
high = mid -
```

```
1;
}
}
}
return -1;
}
```

Dry Run:

- Input: arr = [4, 5, 6, 1, 2, 3], target = 2
- Binary Search: Identify sorted half, move bounds.

Output: 4

5. Square Root of a Number

Problem Statement

Find the square root of a number x with integer precision.

Approach 1: Linear Search

- Idea: Incrementally test numbers until mid pow 2 <= x.
- Time Complexity: $O(\sqrt{x})$
- Space Complexity: O(1)

Pseudocode:

```
int sqrtLinear(int x)
```

```
{
int i = 1;
while (i * i <= x) i++;
return i - 1;
}
```

Dry Run:

Input: x = 10

Traversal: Test 1, 2, 3 (3 pow 2 > 10).

Output: 3

Approach 2: Binary Search

- Idea: Use binary search to find the largest mid such that mid pow 2 <= x.
- Time Complexity: O(log x)
- Space Complexity: O(1)

Pseudocode:

```
int sqrtBinary(int x) {
int low = 0, high = x, result = 0;
while (low <= high)
```

```
{
int mid = low + (high - low)  /2;
if ((long long)mid * mid <= x) {
result = mid;
low = mid + 1;
} else {
high = mid - 1;
}
}
return result;
}
```

Dry Run:

Input: x = 10

Binary Search: mid = 3, 3 pow 2 <= 10.

Output: 3

6. Search in a 2D Matrix

Problem Statement

Given an m x n matrix sorted row-wise and column-wise, search for a

target.

Approach: Row Reduction

- Idea: Start from the top-right corner and eliminate rows or columns based on comparisons.
- Time Complexity: O(m + n)
- Space Complexity: O(1)

Pseudocode:

```
bool searchMatrix(vector>& matrix, int target) {
int rows = matrix.size(), cols = matrix[0].size();
int r = 0, c = cols - 1;
while (r < rows && c >= 0) {
if (matrix[r][c] == target) return true;
else if (matrix[r][c] > target) c--;
else r++;
}
return false;
}
```

Dry Run:

Input: matrix = [[1, 3, 5], [7, 9, 11]], target = 9

Traversal: Move down and

left.

Output: true

7. Koko Eating Bananas

Problem Statement

Koko loves to eat bananas. Given an array piles representing the number of bananas in each pile and an integer h representing the hours she has, find the minimum eating speed (bananas per hour) such that Koko can eat all the bananas within h hours.

Approach 1: Linear Search

- Idea: Try all possible eating speeds from 1 to the maximum pile size. For each speed, calculate the total hours needed.
- Time Complexity: O(max(piles) × n)
- Space Complexity: O(1)

Pseudocode:

```
int minEatingSpeedLinear(vector& piles, int h) {
for (int k = 1; k <= *max_element(piles.begin(), piles.end()); k++) {
int hours = 0;
for (int pile   piles) {
hours += (pile + k - 1)   k;   /Ceiling
```

division

}

if (hours <= h) return k;

}

return -1;

}

Approach 2: Binary Search

- Idea: Use binary search on the range of eating speeds. For each speed, check if it's possible to eat all bananas within h hours.

- Time Complexity: O(n × log(max(piles)))

- Space Complexity: O(1)

Pseudocode:

int minEatingSpeedBinary(vector& piles, int h) {

int low = 1, high = *max_element(piles.begin(), piles.end());

while (low < high) {

int mid = low + (high - low) /2;

int hours = 0;

for (int pile piles) {

hours += (pile + mid - 1) /mid;

}

if (hours <= h) high =

```
mid;
else low = mid + 1;
}
return low;
}
```

Dry Run:

Input: piles = [3, 6, 7, 11], h = 8

Binary search range: [1, 11]

mid = 6, hours = 8 → Possible, adjust range to [1, 6]

Final speed: 4

Output: 4

8. Find the Minimum in a Rotated Sorted Array

Problem Statement

Given a rotated sorted array, find the minimum element.

Approach 1: Linear Search

- Idea: Traverse the array and return the first element that is smaller than the previous element.
- Time Complexity: O(n)
- Space Complexity:

O(1)

Pseudocode:

```
int findMinLinear(vector& nums) {
for (int i = 1; i < nums.size(); i++) {
if (nums[i] < nums[i - 1]) return nums[i];
}
return nums[0];
}
```

Approach 2: Binary Search

- Idea: Use binary search to find the pivot where the array starts to rotate. The minimum element is at the pivot.
- Time Complexity: O(log n)
- Space Complexity: O(1)

Pseudocode:

```
int findMinBinary(vector& nums) {
int low = 0, high = nums.size() - 1;
while (low < high) {
int mid = low + (high - low)  /2;
if (nums[mid] > nums[high]) low = mid + 1;
else high = mid;
}
return
```

nums[low];
}

Dry Run:
Input: nums = [4, 5, 6, 7, 0, 1, 2]
Binary search: Pivot found at index 4

Output: 0

9. Search in a Rotated Sorted Array
Problem Statement
Find the index of a target element in a rotated sorted array.

Approach 1: Linear Search
- Idea: Traverse the array and compare each element with the target.
- Time Complexity: O(n)
- Space Complexity: O(1)

Pseudocode:
int searchLinear(vector& nums, int target) {
for (int i = 0; i < nums.size(); i++) {
if (nums[i] == target) return i;
}
return

-1;
}

Approach 2: Binary Search
- Idea: Use binary search while identifying the sorted half in every iteration.
- Time Complexity: O(log n)
- Space Complexity: O(1)

Pseudocode:
int searchBinary(vector& nums, int target) {
int low = 0, high = nums.size() - 1;
while (low <= high) {
int mid = low + (high - low) /2;
if (nums[mid] == target) return mid;

if (nums[low] <= nums[mid]) { //Left half is sorted
if (target >= nums[low] && target < nums[mid]) high = mid - 1;
else low = mid + 1;
} else { //Right half is sorted
if (target > nums[mid] && target <= nums[high]) low = mid + 1;
else high = mid - 1;
}
}
return

-1;
}

Dry Run:

Input: nums = [4, 5, 6, 7, 0, 1, 2], target = 0
Binary search: Identify the sorted half, adjust bounds

Output: 4

10. Book Allocation
Problem Statement
Given n books and m students, allocate books such that the maximum pages assigned to a student is minimized.

Approach 1: Linear Search
- Idea: Try all possible page limits and check feasibility for each.
- Time Complexity: O(max(pages) × n)
- Space Complexity: O(1)

Pseudocode:
bool isFeasible(vector& books, int students, int maxPages) {
int studentCount = 1, pages = 0;
for (int book books) {
if (pages + book > maxPages)

```
{
studentCount++;
pages = book;
if (studentCount > students) return false;
} else {
pages += book;
}
}
return true;
}

int allocateBooksLinear(vector& books, int students) {
int maxPages = accumulate(books.begin(), books.end(), 0);
for (int limit = *max_element(books.begin(), books.end()); limit <=
maxPages; limit++) {
if (isFeasible(books, students, limit)) return limit;
}
return -1;
}
```

Approach 2: Binary Search

- Idea: Use binary search on the range of page limits and check feasibility for each mid-value.
- Time Complexity: O(n × log(sum(books)))
- Space Complexity:

O(1)

Pseudocode:

```
int allocateBooksBinary(vector& books, int students) {
int low = *max_element(books.begin(), books.end()), high =
accumulate(books.begin(), books.end(), 0);
while (low < high) {
int mid = low + (high - low)  /2;
if (isFeasible(books, students, mid)) high = mid;
else low = mid + 1;
}
return low;
}
```

Dry Run:

Input: books = [10, 20, 30, 40], students = 2

Binary search: Feasibility check for mid-values

Output: 60

11. Search in a 2D Matrix II

Problem Statement

Given a 2D matrix matrix sorted in ascending order row-wise and column-wise, find whether a target exists in the matrix.

Approach 1: Start from Top-Right

Corner

- Idea: Start from the top-right corner of the matrix. Move left if the element is greater than the target, or move down if it is smaller.
- Time Complexity: O(m + n)
- Space Complexity: O(1)

Pseudocode:

```
bool searchMatrix(vector>& matrix, int target) {
int row = 0, col = matrix[0].size() - 1;
while (row < matrix.size() && col >= 0) {
if (matrix[row][col] == target) return true;
else if (matrix[row][col] > target) col--;
else row++;
}
return false;
}
```

Approach 2: Binary Search on Rows

- Idea: For each row, perform binary search.
- Time Complexity: O(m × log(n))
- Space Complexity: O(1)

Pseudocode:

```
bool binarySearch(vector& row, int target) {
int low = 0, high = row.size() -
```

```
1;
while (low <= high) {
int mid = low + (high - low)  /2;
if (row[mid] == target) return true;
else if (row[mid] > target) high = mid - 1;
else low = mid + 1;
}
return false;
}

bool searchMatrixBinary(vector>& matrix, int target) {
for (auto& row  matrix) {
if (binarySearch(row, target)) return true;
}
return false;
}
```

Dry Run:

Input: matrix = [[1, 4, 7], [2, 5, 8], [3, 6, 9]], target = 5

Start from top-right: Compare, move → Found at (1, 1)

Output: true

12. Median of Two Sorted

Arrays

Problem Statement

Find the median of two sorted arrays.

Approach: Binary Search on Smaller Array

- Idea: Partition the arrays into two halves and adjust the partition using binary search.
- Time Complexity: O(log(min(m, n)))
- Space Complexity: O(1)

Pseudocode:

```
double findMedianSortedArrays(vector& nums1, vector& nums2) {
if (nums1.size() > nums2.size()) swap(nums1, nums2);
int x = nums1.size(), y = nums2.size();
int low = 0, high = x;

while (low <= high) {
int partitionX = (low + high)  /2;
int partitionY = (x + y + 1)  /2 - partitionX;

int maxX = (partitionX == 0)  INT_MIN  nums1[partitionX - 1];
int minX = (partitionX == x)  INT_MAX  :
```

```
nums1[partitionX];


int maxY = (partitionY == 0)  INT_MIN  nums2[partitionY - 1];
int minY = (partitionY == y)  INT_MAX  nums2[partitionY];

if (maxX <= minY && maxY <= minX) {
if ((x + y) % 2 == 0) return (max(maxX, maxY) + min(minX, minY))  2.0;
else return max(maxX, maxY);
} else if (maxX > minY) high = partitionX - 1;
else low = partitionX + 1;
}
return -1;
}
```

13. Aggressive Cows

Problem Statement

Place cows in stalls such that the minimum distance between them is maximized.

Approach: Binary Search on Minimum Distance

- Idea: Use binary search on the distance range and check feasibility for each mid-value.
- Time Complexity: O(n × log(max(stalls) - min(stalls)))
- Space Complexity: